

Code Manual: Project 11

Nuri van Dijk

26 March 2026

PDF VERSION	Third version
CODE VERSION	0.6.2
REPOSITORY LINK	https://github.com/AineMedias/CQM

Abstract

This program is designed to take three potentials and create H -matrices from them by embedding them in an infinite square well of size a using the method outlined in the 2013 paper by Jugdutt[1], in order to determine the energy eigenvalues arising from said potentials. Depending on user input, the script may additionally generate graphs that show critical coefficients as a function.

In order to allow this program to do so with some efficiency, it is assumed that the potentials in question are central in three dimensions and the Schrödinger equation for this potential is separable into radial and axial components - as such, this algorithm is not expected to run as well on potentials that are symmetric in three dimensions but not spherically, such as the quantum dot.

1 Mathematical model

The centerpiece of this program revolves around the `eigenvalues()` function, which uses a predefined potential $V(r)$ and upper bound n_{max} to determine the two-dimensional Hamiltonian matrix H_{nm} according to the formula outlined in the paper by Jugdutt[1]:

$$H_{nm} = \delta_{nm} E_n^0 + \int_0^a dr \sin\left(\frac{n\pi r}{a}\right) \left[V(r) + \frac{\hbar^2 \ell(\ell+1)}{2m_0 r^2} \right] \sin\left(\frac{m\pi r}{a}\right) \quad (1)$$

where $E_n^0 = \frac{\alpha^2 m_e c^2}{2n^2} \approx \frac{13.6}{n^2}$ eV is the Rydberg energy as a function of the principal quantum number n , e is the elementary charge, m_0 is the electronic mass, and a is the size of the embedding infinite square well.

Since the program has its predefined SI constants expressed in atomic units, where $\hbar$, e and m_0 are unitary, and the permittivity $\epsilon_0 = \frac{1}{4\pi}$, Equation 1 reduces to

$$H_{nm} = \delta_{nm}E_n^0 + \int_0^a dr \sin\left(\frac{n\pi r}{a}\right) \left[V(r) + \frac{\ell(\ell+1)}{2r^2} \right] \sin\left(\frac{m\pi r}{a}\right) \quad (2)$$

where a is now defined in units of the Bohr radius a_0 , and both the Rydberg energy and the Hamiltonian eigenvalues,

$$\Sigma(H_{nm}c_n) = E_n c_m \quad (3)$$

are defined in units of the Hartree energy $E_h = \frac{\hbar^2}{m_0 a_0^2} \approx 27.211$ eV.

2 Algorithm overview

Each algorithm can be divided into two distinct code blocks: one that handles core computation and another to handle critical value determination. Whether all blocks are run instead of only the former is dependent on the quantum system to be examined.

2.1 Core computation

This step involves the determination of the Hamiltonian matrix H_{nm} itself - given the variable nature of both $V(r)$ and ℓ within Equation 2, the core function `eigenvalues()` lists both $V(r)$ and ℓ , alongside the embedding potential size a and for each evaluated n and m the integration step was to be performed using a SciPy `integrate.quad()` function. Then, in order to evaluate the entirety of H_{nm} , the aforementioned function was encased in a nested loop, looping over m first, then n , up to a predefined limit n_{max} - with the eigenvalues of H_{nm} determined through a NumPy `linalg.eigvals()` function. This function returns both the eigenvalues E_n and one-dimensional eigenvectors c_n , of which only the former is of interest.

2.2 Critical value determination

Then, taking the function to determine the H_{nm} matrix and its associated E_n given a defined $V(r)$, determining the critical values associated with a particular potential, such as the critical screening coefficient μ_c for the Yukawa potential, requires that $V(r)$ be altered mid-execution. As such, the choice was made to loop the above-mentioned function over the relevant variables such as the screening coefficient μ , and both define $V(r)$ and run `eigenvalues()` with the current $V(r)$ mid-loop.

3 Software interface and code execution

Any computer that a user wants to have this program executed on requires Python version 3.10 or later to be installed within any virtual environment, as below this version parts of the Python file's main function that use the match-case system will not work and raise exceptions accordingly. Furthermore, assuming the virtual environment was freshly made to run this specific program, the matplotlib, numpy, scipy and tqdm packages must be installed as well, either through `pip install` or, if the Anaconda distribution is currently in use, through `conda install` within a PowerShell instance.

If all required packages are installed, the code can simply be run through the command `python3 centralpotentials.py`, or `& envpath "filepath" in Anaconda`, where `envpath` represents the path leading to the virtual environment's `python.exe` file, and `filepath` represents the path leading to the downloaded `centralpotentials.py` file.

In both cases, before execution of the eigenvalues function or loop containing said function begins, the user is asked to choose a quantum system to examine by inserting a string representing the system in question: the supported strings are `h-atom` representing the Coulomb potential, `fin-well` representing the finite spherical well and `yukawa` for the Yukawa potential. If any other string is inserted, a `ValueError` is raised.

Next, the user is asked whether they want to examine the potential at a given value or execute the default function on these: they are supposed to type in respectively 1 or 0 before continuing on, otherwise another `ValueError` is raised. Dependent on the selected potential, more variables might be required as input, such as the screening constant μ for the Yukawa potential or the well depth V_0 and width b for the finite spherical well.

Afterwards, code execution begins. In case that a peculiar potential is to be examined or the selected potential is `h-atom`, the program will run `eigenvalues()` once to generate a graph where the energy eigenvalues in Hartree units ($E_h = 27.211$ eV) are defined as a function of n . For the Yukawa potential specifically, the lowest energy eigenvalue E_0 is also returned in Hartree units.

On the other hand, if the selected potential is `fin-well` or `yukawa` and the default function variable is set to 0, the program will instead loop over respectively V_0 and b , or over V in Hartree units, and instead returns a list for `fin-well`, or the critical coefficient μ_c for `yukawa`.

4 Benchmark potential tests and results

4.1 Hydrogen atom

In order to test the basic functionality of this numeric algorithm, the Coulomb potential was examined, and tested to see whether the algorithm would reproduce the results from Jugdutt. When the potential in question, defined in atomic units as

$$V(r) = \frac{1}{r} \quad (4)$$

is inserted into equation 2, and the resulting Hamiltonian eigenvalues for $n_{max} = 200$ plotted as a function of n , the acquired figure can be seen as Figure 1.

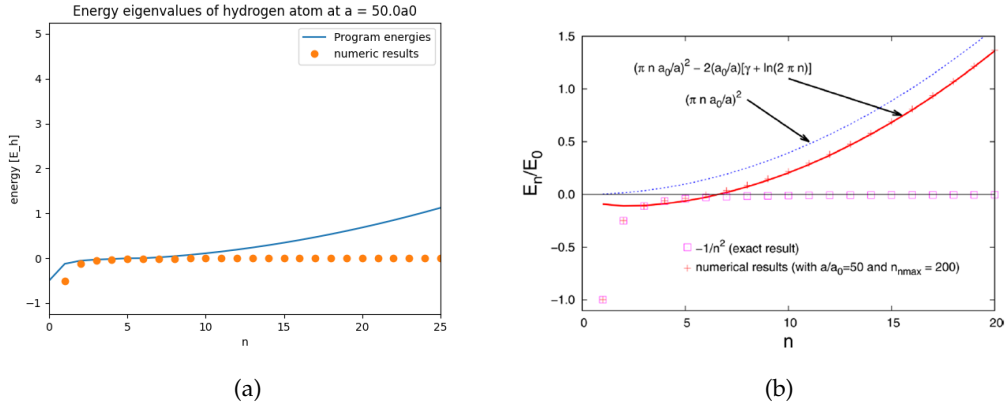


Figure 1: Computed energy eigenvalues of the Coulomb potential system with $n_{max} = 200$ and $a = 50a_0$, compared to numeric results (a) and the expected results. Subfigure (b) reproduced from [1].

Comparing Figures 1a and 1b with one another reveals that both lists of computed values diverge at similar values of n , as initially expected.

4.2 Yukawa potential

A similar thing was attempted with the Yukawa potential, defined as

$$V_y(r) = -A \frac{e^{-\mu r/a_0}}{r} \quad (5)$$

where A is a constant representing the electronic interaction strength and μ represents the screening coefficient, in order to figure out the critical coefficient μ_c beyond which no existing eigenstate remains bound.

Due to existing performance issues concerning the full algorithm, the decision was made to examine the Yukawa potential at a smaller range of μ , ranging from $\mu = 0.9$ to 1.25 instead of $\mu = 0.9$ to 1.5 . When this altered algorithm was in turn run, the algorithm stops at $\mu = 1.0$, suggesting a critical coefficient value of $\mu_c = 0.99 \pm$

0.01. This particular value for μ_c conflicts both with previous findings suggesting $\mu_c \approx 1.19$ [1], as well as subsequent manual analyses at both $\mu = 0.99$ and $\mu = 1.00$ suggesting $\mu_c > 1.00$, as seen in Figure 2.

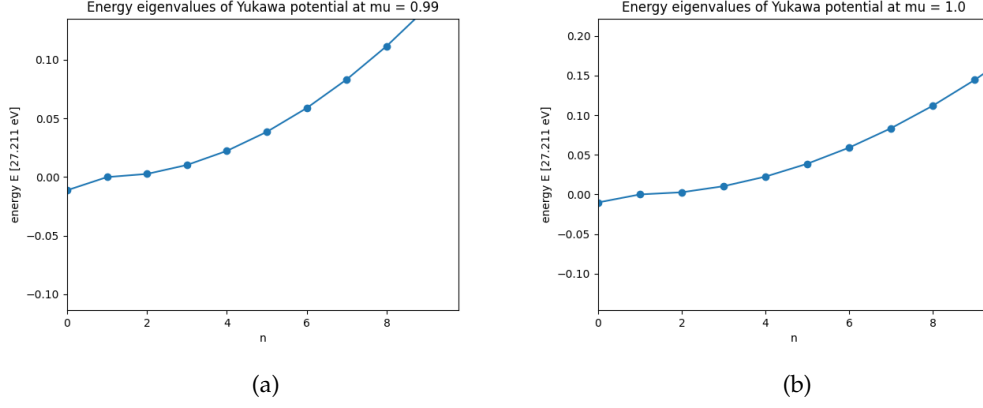


Figure 2: Computed energy eigenvalues of the Yukawa potential system as a function of n , at $\mu = 0.99$ and $\mu = 1.00$ respectively. From these two systems, the lowest energy eigenstates of each were determined to be $E_0 = -0.012E_h$ and $E_0 = -0.010E_h$, meaning that by definition $\mu_c > 1.00$.

5 Performance

Since the `eigenvalues()` function features within a nested loop a `integrate.quad()` function run over a definite interval, and thus utilizes a 21-point Gauss-Kronrod quadrature function of order $2n + 1$ [2], the overall structure requiring for-loops suggests that a runtime of order $O(n^3)$, scaling with both a and n_{max} , applies to `eigenvalues()` itself, while a runtime of order $O(n^4)$ applies to the single-loop critical potential algorithm, and a runtime of order $O(n^5)$ applies to the nested loop of the finite spherical well algorithm.

This is best observed with the execution time for each of the three algorithms: whereas `eigenvalues()`, assuming $n_{max} = 200$ and $a = 50a_0$, requires up to three minutes, the critical value determinations for the Yukawa potential and the finite spherical well require up to twenty minutes and several hours respectively to complete.

6 References

- [1] BA Jugdutt and F Marsiglio. "Solving for three-dimensional central potentials using numerical matrix methods". In: *American Journal of Physics* 81.5 (2013), pp. 343–350.

- [2] T. Oliphant, P. Peterson, and E. Jones. *SciPy*. <https://github.com/scipy/scipy>. 2001.